

# Laboratório 05

ra: 175886

Joao Vitor Saes Casagrande

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(leaflet)
```

```
library(lubridate)
```

```
library(htmlwidgets)
```

```
# =====
```

```
# Função Principal: analisa_aeronave
```

```
# =====
```

```
#' Analisa os trajetos de uma aeronave ao longo do ano.
```

```
#'
```

```
#' @param tail_number Código identificador do avião (ex: "N431WN").
```

```
#' @param arquivo Caminho completo para o arquivo de voos (ex: "dados/flights.csv.zip").
```

```
#' @return Uma lista nomeada contendo a tabela limpa (tidy) e o mapa leaflet.
```

```
analisa_aeronave <- function(tail_number, arquivo) {
```

```
# -----
```

```
# 1. Leitura de Dados em Partes (Chunked) e Filtragem
```

```
# -----
```

```

# Definimos uma função de callback que filtra o chunk (pedaço) de dados
# mantendo apenas as linhas referentes à aeronave desejada.
funcao_filtro <- function(pedaco_de_dados, indice_posicao) {
  pedaco_de_dados %>%
    dplyr::filter(TAIL_NUMBER == tail_number)
}

# Lemos o arquivo grande por partes. O tamanho do chunk é de 100.000 linhas.
# Isso previne que a memória RAM do computador seja sobrecarregada.
dados_brutos_aeronave <- readr::read_csv_chunked(
  file = arquivo,
  callback = readr::DataFrameCallback$new(funcao_filtro),
  chunk_size = 100000,
  show_col_types = FALSE
)

# -----
# 2. Limpeza, Ordenação Temporal e Formatação (Tidy Data)
# -----

tabela_tidy_voos <- dados_brutos_aeronave %>%
  # Filtra voos cancelados ou sem horário de partida para evitar erros no mapa
  dplyr::filter(!is.na(DEPARTURE_TIME)) %>%
  dplyr::mutate(
    # Cria uma coluna de data unindo YEAR, MONTH e DAY
    DATA_VOO = lubridate::make_date(YEAR, MONTH, DAY),

    # DEPARTURE_TIME geralmente vem como número inteiro (ex: 1130 para 11:30).
    # Formatamos adicionando zeros à esquerda para garantir 4 caracteres.
    HORA_FORMATADA = stringr::str_pad(DEPARTURE_TIME, width = 4, pad = "0"),
    HORA_FORMATADA = paste0(substr(HORA_FORMATADA, 1, 2), ":", substr(HORA_FORMATADA, 3, 4)),

    # Combina Data e Hora em um único formato datetime
    DATA_HORA_PARTIDA = lubridate::ymd_hm(paste(DATA_VOO, HORA_FORMATADA)),

    # Calcula a velocidade média do trajeto: Distância (milhas) / Tempo (horas)
    VELOCIDADE_MEDIA_MPH = DISTANCE / (AIR_TIME / 60)
  ) %>%
  # Ordena cronologicamente para garantir que o trajeto seja linear no tempo
  dplyr::arrange(DATA_HORA_PARTIDA)

# -----
# 3. Cruzamento com Coordenadas dos Aeroportos

```

```

# -----
# Infere-se que "airports.csv" está na mesma pasta do arquivo de voos
diretorio_dados <- dirname(arquivo)
arquivo_aeroportos <- file.path(diretorio_dados, "airports.csv")

# Lê o arquivo de aeroportos
tabela_aeroportos <- readr::read_csv(arquivo_aeroportos, show_col_types = FALSE) %>%
  dplyr::select(IATA_CODE, LATITUDE, LONGITUDE)

# Adiciona latitude e longitude da Origem e do Destino na tabela de voos
tabela_com_coordenadas <- tabela_tidy_voos %>%
  dplyr::left_join(tabela_aeroportos, by = c("ORIGIN_AIRPORT" = "IATA_CODE")) %>%
  dplyr::rename(LAT_ORIGEM = LATITUDE, LON_ORIGEM = LONGITUDE) %>%

  dplyr::left_join(tabela_aeroportos, by = c("DESTINATION_AIRPORT" = "IATA_CODE")) %>%
  dplyr::rename(LAT_DESTINO = LATITUDE, LON_DESTINO = LONGITUDE) %>%

# Remove voos que porventura tenham aeroportos sem coordenadas registradas
dplyr::filter(!is.na(LAT_ORIGEM) & !is.na(LAT_DESTINO))

# -----
# 4. Construção do Mapa (Leaflet)
# -----
# Inicia o mapa base com tiles simples e limpos (CartoDB Positron)
mapa_trajeto_anual <- leaflet::leaflet() %>%
  leaflet::addProviderTiles(leaflet::providers$CartoDB.Positron)

# Itera sobre os voos em ordem cronológica para desenhar os trajetos
# O loop é ideal aqui pois Leaflet lida com a espessura de linhas segmento a segmento.
for(indice in seq_len(nrow(tabela_com_coordenadas))) {

  voo_atual <- tabela_com_coordenadas[indice, ]

  # Define a espessura da linha com base na velocidade média.
  # Valores divididos por 100 para manter a estética visual do mapa proporcional.
  espessura_estatistica <- ifelse(is.na(voo_atual$VELOCIDADE_MEDIA_MPH),
    1,
    voo_atual$VELOCIDADE_MEDIA_MPH / 100)

  # Cria uma janela de informação (popup) para cada segmento de voo
  info_voo_html <- paste0(
    "<b>Voo:</b> ", voo_atual$ORIGIN_AIRPORT, " ", voo_atual$DESTINATION_AIRPORT, "<br>",

```

```

    "<b>Data/Hora:</b> ", voo_atual$DATA_HORA_PARTIDA, "<br>",
    "<b>Velocidade Média:</b> ", round(voo_atual$VELOCIDADE_MEDIA_MPH, 1), " mph<br>",
    "<b>Distância:</b> ", voo_atual$DISTANCE, " milhas"
  )

  # Adiciona a linha (trajeto) no mapa
  mapa_trajeto_anual <- mapa_trajeto_anual %>%
    leaflet::addPolylines(
      lng = c(voo_atual$LON_ORIGEM, voo_atual$LON_DESTINO),
      lat = c(voo_atual$LAT_ORIGEM, voo_atual$LAT_DESTINO),
      weight = espessura_estatistica,
      color = "#2c3e50",          # Cor das rotas (Azul escuro)
      opacity = 0.5,              # Transparência para sobreposição visível
      popup = info_voo_html
    )
}
resultado_final <- list(
  tabela = tabela_tidy_voos,
  mapa = mapa_trajeto_anual
)

return(resultado_final)
}

```

```

resultado_voos <- analisa_aeronave(
  tail_number = "N431WN",
  arquivo = "flights.csv"
)
resultado_voos

```

\$tabela

# A tibble: 1,926 x 35

|   | YEAR  | MONTH | DAY   | DAY_OF_WEEK | AIRLINE | FLIGHT_NUMBER | TAIL_NUMBER |
|---|-------|-------|-------|-------------|---------|---------------|-------------|
|   | <dbl> | <dbl> | <dbl> | <dbl>       | <chr>   | <dbl>         | <chr>       |
| 1 | 2015  | 1     | 1     | 4           | WN      | 1762          | N431WN      |
| 2 | 2015  | 1     | 1     | 4           | WN      | 3564          | N431WN      |
| 3 | 2015  | 1     | 1     | 4           | WN      | 3564          | N431WN      |
| 4 | 2015  | 1     | 1     | 4           | WN      | 3564          | N431WN      |
| 5 | 2015  | 1     | 2     | 5           | WN      | 1793          | N431WN      |
| 6 | 2015  | 1     | 2     | 5           | WN      | 1793          | N431WN      |
| 7 | 2015  | 1     | 2     | 5           | WN      | 1793          | N431WN      |
| 8 | 2015  | 1     | 2     | 5           | WN      | 1793          | N431WN      |

```

 9  2015      1      2          5 WN          1793 N431WN
10  2015      1      2          5 WN          3144 N431WN
# i 1,916 more rows
# i 28 more variables: ORIGIN_AIRPORT <chr>, DESTINATION_AIRPORT <chr>,
#   SCHEDULED_DEPARTURE <chr>, DEPARTURE_TIME <chr>, DEPARTURE_DELAY <dbl>,
#   TAXI_OUT <dbl>, WHEELS_OFF <chr>, SCHEDULED_TIME <dbl>, ELAPSED_TIME <dbl>,
#   AIR_TIME <dbl>, DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <dbl>,
#   SCHEDULED_ARRIVAL <chr>, ARRIVAL_TIME <chr>, ARRIVAL_DELAY <dbl>,
#   DIVERTED <dbl>, CANCELLED <dbl>, CANCELLATION_REASON <chr>, ...

```

```
$mapa
```

```

Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()

```

```
[1] "2026-09-16 13:05:14 -03"
```